

Tendências de Motores de Jogos

Semana 2

Character, Movimentação e Enhanced Input

Disciplina: Tendências de Motores de Jogos (IN46A)

Instituição: IFMS — Campus Dourados

Curso: Tecnologia em Jogos Digitais

Módulo 1 — Aprender a Ferramenta

IFMS · Tecnologia em Jogos Digitais · Tendências de Motores de Jogos

Objetivos da Semana

OBJETIVOS

- Distinguir Pawn de Character e o papel do Character Movement Component
- Configurar o `BP_Player` como Character controlável no nível de teste
- Compreender Enhanced Input como desacoplamento entre dispositivo e intenção
- Implementar movimentação, câmera e uma Input Action própria

Como a semana está organizada



Encontro 1

Pawn x Character + Character Movement Component + BP_Player no nível de teste



Encontro 2

Enhanced Input (Input Actions, Mapping Context) + desafio de nova ação

Tendências de Motores de Jogos

ENCONTRO 1

Pawn, Character e Movimentação

IFMS · Tecnologia em Jogos Digitais · Tendências de Motores de Jogos

Tendências de Motores de Jogos



Por que não controlar diretamente o Actor da Semana 1?

Pense no que seria necessário reimplementar manualmente: input, caminhada, salto, colisão com o chão.

IFMS · Tecnologia em Jogos Digitais · Tendências de Motores de Jogos

Pawn x Character

- **Pawn** — Actor que pode ser possuído por um Controller, sem locomoção pronta
- **Character** — especialização de Pawn com Character Movement Component embutido
- Character resolve caminhada, corrida, salto, natação, voo e colisão com o terreno



DICA

O mesmo problema de locomoção que toda engine de terceira pessoa precisa resolver.

Tendências de Motores de Jogos

De Actor a Character



IFMS · Tecnologia em Jogos Digitais · Tendências de Motores de Jogos

Locomoção: Unreal × Unity

Unreal

Character já embute Character Movement
Component completo por padrão

Unity

GameObject +
CharacterController/Rigidbody + script de
movimento próprio

Demonstração: criando o BP_Player

O professor cria `BP_Player` como subclasse de Character (não Actor, não Pawn) e ajusta parâmetros de movimento.

O que muda: velocidade de caminhada e altura de salto perceptivelmente diferentes do padrão do template.

Imagem sugerida

Objetivo: mostrar o painel de Components do BP_Player recém-criado.

Enquadramento: editor de Blueprint com o painel de Components à esquerda.

Elementos presentes: Capsule Component, Arrow Component, Mesh Component e a entrada do Character Movement Component.

Destaque visual: contorno ao redor da entrada do Character Movement Component, ausente no BP_Testecomposicao da Semana 1.

Legenda sugerida: "O Character já nasce com Capsule, Mesh e Character Movement Component — nada disso precisa ser montado manualmente."

Arquitetura: BP_Player no projeto

- Blueprints/Characters/BP_Player — Blueprint central do projeto
- Default Pawn Class do projeto passa a apontar para BP_Player
- Base sobre a qual Input, Interaction e outros sistemas serão anexados

NA INDÚSTRIA

O mesmo Blueprint concentra, ao longo do semestre, locomoção, câmera, input e Components de gameplay do jogador.

Boas práticas

✓ BOAS PRÁTICAS

- Partir de Character sempre que houver locomoção física e controle direto
- Reservar Actor puro para objetos do mundo sem controle do jogador
- Ajustar parâmetros de movimento em pequenos incrementos, testando a cada mudança

Laboratório do Encontro 1

Criar o `BP_Player` como subclasse de `Character`, ajustar `Max Walk Speed` e `Jump Z Velocity`, e posicioná-lo controlável no `Map_Exploration`.

OBJETIVOS

Critério de sucesso: `BP_Player` se move com o controle padrão, com velocidade e salto perceptivelmente diferentes do template.

Fechando o Encontro 1

- Pawn habilita controle; Character adiciona locomoção pronta
- BP_Player criado, ajustado e controlável no nível de teste
- Próximo encontro: substituir o controle padrão por Enhanced Input

Tendências de Motores de Jogos

ENCONTRO 2

Enhanced Input

IFMS · Tecnologia em Jogos Digitais · Tendências de Motores de Jogos

Tendências de Motores de Jogos



**O que acontece se a lógica de gameplay
checar diretamente "a tecla W foi
pressionada"?**

Pense em trocar de teclado para controle, remapear teclas ou dar suporte a acessibilidade.

IFMS · Tecnologia em Jogos Digitais · Tendências de Motores de Jogos

Enhanced Input: quatro peças

- **Input Action** — intenção abstrata ("mover", "olhar", "pular")
- **Input Mapping Context** — mapeamento entre dispositivo e Input Action
- **Trigger** — quando o valor é considerado acionado
- **Modifier** — transforma o valor bruto (inversão, zona morta, escala)

DICA

A lógica de gameplay nunca deveria depender de qual tecla foi pressionada.

Tendências de Motores de Jogos

Do dispositivo à ação no mundo



IFMS · Tecnologia em Jogos Digitais · Tendências de Motores de Jogos

Enhanced Input × Input System

Unreal

Input Mapping Contexts combináveis e priorizáveis em runtime

Unity

Action Maps dentro de um único Input Actions Asset, ativados via código

Demonstração: criando IA_Move, IA_Look e IA_Jump

O professor cria três Input Actions e um Input Mapping Context (`IMC_Player`), mapeando WASD, mouse e barra de espaço.

Por que: separar a intenção (mover, olhar, pular) do dispositivo que a aciona.

Demonstração: conectando ao BP_Player

Add Mapping Context no BeginPlay, depois os nós Enhanced Input Action conectados a Add Movement Input, Add Controller Yaw/Pitch Input e Jump.

Resultado esperado: controle exclusivamente via `IMC_Player`, sem resquício do template.

Imagem sugerida

Objetivo: mostrar o Event Graph do BP_Player com os nós de Enhanced Input conectados às funções de movimento.

Enquadramento: captura do Event Graph, centralizada nos nós Enhanced Input Action Move/Look/Jump.

Elementos presentes: nó "Add Mapping Context" no Event BeginPlay; nós "Enhanced Input Action" para IA_Move, IA_Look e IA_Jump; conexões para Add Movement Input, Add Controller Yaw/Pitch Input e Jump.

Destaque visual: o fluxo desde o evento de input abstrato até a chamada concreta de movimentação.

Legenda sugerida: "Do dispositivo físico à ação no mundo: o fluxo completo do Enhanced Input no BP_Player."

Boas práticas

✓ BOAS PRÁTICAS

- Nomear Input Actions pela intenção, nunca pelo dispositivo (`IA_Sprint` , não `IA_TecLaShift`)
- Organizar os nós de input em uma Comment Box separada no Event Graph
- Cada tecla mapeada a uma única intenção dentro do mesmo Mapping Context

Laboratório do Encontro 2

Criar `IA_Move`, `IA_Look`, `IA_Jump` e `IMC_Player`, e conectar ao `BP_Player`, substituindo o controle padrão do template.

OBJETIVOS

Critério de sucesso: movimentação, câmera e salto funcionais exclusivamente via Enhanced Input.

Desafio: uma nova Input Action

Escolher entre correr, agachar ou uma variação de salto, e implementar com liberdade de Trigger e Modifier.

ATENÇÃO

Nomear pela intenção, nunca pela tecla. Sem gabarito único.

Resumo da Semana 2

- Character = Pawn + Character Movement Component pronto
- BP_Player controlável no nível de teste
- Enhanced Input desacopla dispositivo de intenção de jogo
- Cada grupo implementou uma Input Action própria

Próximos passos



DICA

Na Semana 3, o material simples e o terreno via Landscape serão aplicados ao mesmo nível em que o BP_Player já se move — e a turma fará o primeiro build empacotado da disciplina.

Leitura recomendada: Enhanced Input in Unreal Engine · Gameplay Framework in Unreal Engine (Epic Games Documentation).